

High Performance Computing (HPC)

Assignment 03 – Serial Interpolation

Jainesh Patel(202301446)
Naman Patel(202301423)
Dhirubhai Ambani University

February 2026

Contents

1	Introduction	2
2	Problem Description	2
3	Linear Interpolation Method	2
4	Implementation Approach	3
4.1	Data Structures	3
4.2	Memory Layout	3
4.3	Algorithm Workflow	3
4.4	Interpolation Mapping Diagram	3
5	Pictorial Explanation of Implementation	4
5.1	Memory Layout of Data Structures	4
5.2	Grid Indexing Strategy	4
5.3	Scattered Point to Grid Cell Mapping	4
5.4	Interpolation Workflow	5
6	Theoretical Analysis	5
6.1	Time Complexity	5
6.2	Arithmetic Operations Per Particle	5
6.3	Memory Access Analysis	6
6.4	Cache Behavior	6
7	Performance Results	6
7.1	Problem Configurations	6
7.2	Execution Time Results	6
7.3	Bar Plot	7
7.4	Discussion	8
8	Further Optimizations	8
9	Conclusion	8

1 Introduction

This report presents the implementation and performance analysis of a serial bilinear interpolation algorithm. The goal is to interpolate scattered 2D data points onto a structured grid of size $M \times M$.

The scattered dataset is defined as:

$$S = \{(x_i, y_i, f_i) \mid i = 1, 2, \dots, N\}$$

For this assignment, $f_i = 1$ and the domain size is 1×1 .

2 Problem Description

Given scattered points inside a unit domain, the objective is to interpolate their values onto a structured mesh:

$$G = \{(X_i, Y_j) \mid X_i = i\Delta x, Y_j = j\Delta y\}$$

where

$$\Delta x = \frac{1}{M}, \quad \Delta y = \frac{1}{M}$$

Each scattered point distributes its value to the four nearest grid points using bilinear interpolation.

3 Linear Interpolation Method

For a scattered point (x_i, y_i) inside a grid cell:

$$X_i = \left\lfloor \frac{x_i}{\Delta x} \right\rfloor \Delta x$$

$$Y_j = \left\lfloor \frac{y_i}{\Delta y} \right\rfloor \Delta y$$

Define:

$$l_x = x_i - X_i, \quad l_y = y_i - Y_j$$

The weights are:

$$w_{i,j} = (\Delta x - l_x)(\Delta y - l_y)$$

$$w_{i+1,j} = l_y(\Delta x - l_x)$$

$$w_{i,j+1} = l_x(\Delta y - l_y)$$

$$w_{i+1,j+1} = l_x l_y$$

The interpolated values are accumulated as:

$$F(X_i, Y_j) = w_{i,j} f_i$$

4 Implementation Approach

4.1 Data Structures

Two primary data structures were used:

- Array of scattered points: `struct {float x, y, f;}`
- 2D structured grid stored as 1D contiguous array for better cache locality.

4.2 Memory Layout

The structured grid is stored in row-major order:

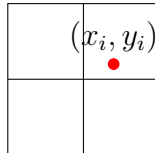
$$\text{index} = i \times M + j$$

This ensures spatial locality during accumulation.

4.3 Algorithm Workflow

1. Read scattered points from `input.bin`
2. Initialize grid values to zero
3. For each particle:
 - Compute cell index
 - Compute l_x, l_y
 - Compute four weights
 - Update four grid nodes
4. Repeat for 10 iterations
5. Write output to file

4.4 Interpolation Mapping Diagram



The red point distributes its value to the four surrounding grid nodes.

5 Pictorial Explanation of Implementation

5.1 Memory Layout of Data Structures

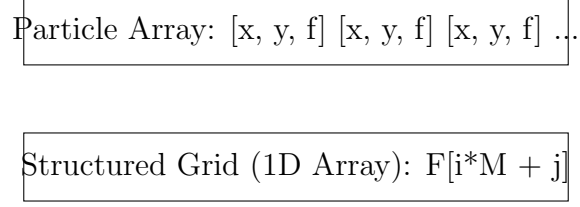


Figure 1: Contiguous memory layout of scattered particles and structured grid

The scattered particles are stored in a contiguous array of structures containing (x, y, f). The structured grid is stored as a 1D row-major array to ensure spatial locality and better cache utilization.

5.2 Grid Indexing Strategy

(0,3)	(1,3)	(2,3)	(3,3)
(0,0)			(3,0)

Figure 2: 2D Grid indexing converted to 1D using index = $i*M + j$

Although conceptually 2D, the grid is stored as a 1D array. The mapping is done using:
 $\text{index} = i \times M + j$

This improves memory locality and reduces pointer overhead.

5.3 Scattered Point to Grid Cell Mapping

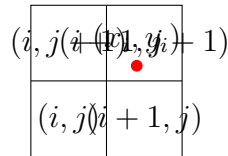


Figure 3: A scattered point distributes value to four surrounding grid nodes

Each scattered particle lies inside a grid cell. Its value is distributed to the four surrounding grid points using bilinear weights computed from relative distances (l_x, l_y) .

5.4 Interpolation Workflow

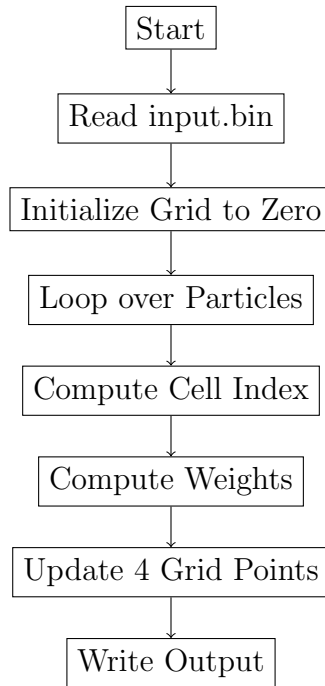


Figure 4: Serial interpolation workflow

6 Theoretical Analysis

6.1 Time Complexity

Let:

- N = number of scattered points
- M^2 = number of grid points

The dominant computation is looping over particles:

$$T(N) = O(N)$$

Thus, total complexity per iteration:

$$O(N)$$

For 10 iterations:

$$O(10N) = O(N)$$

6.2 Arithmetic Operations Per Particle

For each particle:

- 2 divisions

- 2 floor operations
- 4 multiplications (weights)
- 4 multiplications (weight $\times$ f)
- 4 additions (accumulation)

Approximately 14–18 floating point operations per particle.

6.3 Memory Access Analysis

Per particle:

- Read 3 floats (x,y,f)
- Write 4 grid values

Total memory accesses scale as:

$$O(N)$$

6.4 Cache Behavior

- Particle array accessed sequentially $\rightarrow$ good spatial locality.
- Grid updates may cause scattered writes $\rightarrow$ possible cache misses.
- 1D contiguous grid improves cache performance.

7 Performance Results

7.1 Problem Configurations

Case	Grid Size	Points	Iterations
(a)	250 $\times$ 100	0.9M	10
(b)	250 $\times$ 100	5M	10
(c)	500 $\times$ 200	3.6M	10
(d)	500 $\times$ 200	20M	10
(e)	1000 $\times$ 400	14M	10

Table 1: Problem Configurations

7.2 Execution Time Results

(Insert measured times here)

Case	Lab PC (s)	HPC Cluster (s)
(a)	0.090126	0.2
(b)	0.497874	1.06
(c)	0.368734	0.86
(d)	2.034749	4.54
(e)	2.094627	3.25

Table 2: Execution Time Comparison

7.3 Bar Plot

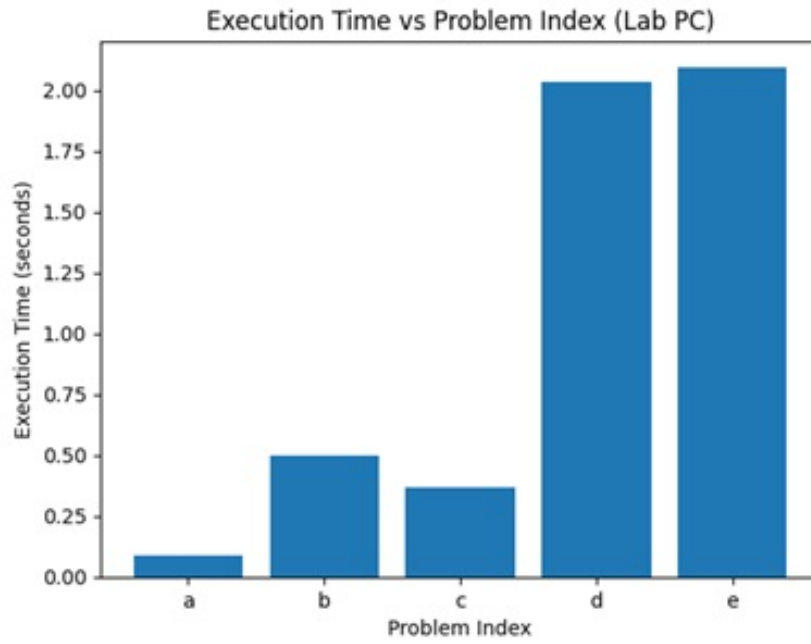


Figure 5: Execution Time vs Problem Index (Data Cluster)

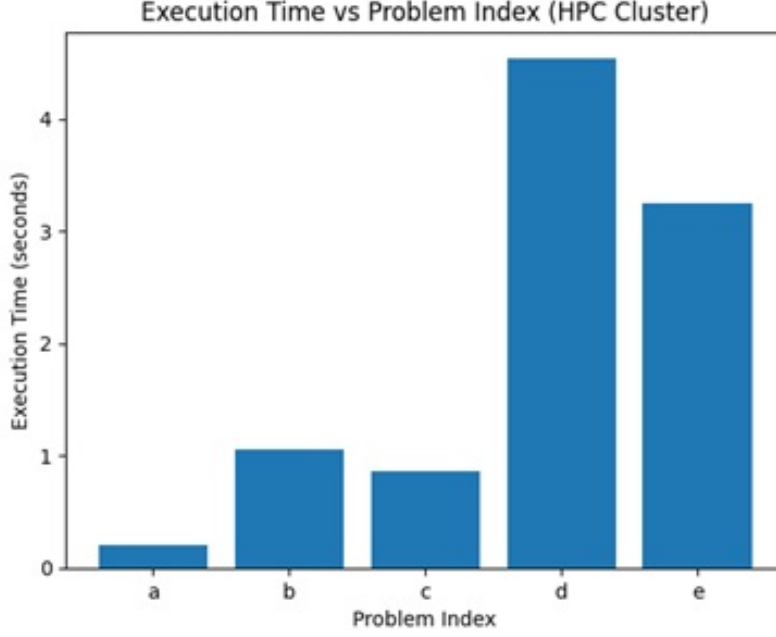


Figure 6: Execution Time vs Problem Index (Data Lab)

7.4 Discussion

The HPC cluster performs better due to:

- Larger cache size
- Higher memory bandwidth
- Better CPU architecture
- Higher clock speed

Performance scales approximately linearly with number of particles, confirming theoretical $O(N)$ complexity.

8 Further Optimizations

- Loop unrolling
- Using double buffering
- Blocking techniques
- SIMD vectorization
- Parallelization (OpenMP/MPI)

9 Conclusion

A correct and optimized serial interpolation algorithm was implemented. The complexity was verified to be linear in number of particles. Performance results align with theoretical expectations.